PREDICTING NEXT HOUR WASTE HEAT AVAILABILITY IN A DATA CENTER

NKEZI EMMANUEL AKOFOR (202674785) nna56@mail.aub.edu PROJECT PROPOSAL
American University of Beirut Introduction to Machine Learning MECH 658

1. Problem and motivation

Can recent computing power and coolant measurements improve next hour average waste heat forecasts beyond past heat alone? The project uses Frontier's 2023 operating data and chronological evaluation. Operators could use forecasts to plan heat pumps, storage, or backup heating. The target is source side thermal power; delivered heat also depends on temperatures, equipment, and demand.

The practical goal is **at least 10% lower test MAE than persistence**. Separately, operating variable models must outperform heat only models on identical histories and target hours to demonstrate added predictive value.

2. Impact

Success would identify useful operating measurements and whether fewer inputs suffice. Failure would show that simple predictors remain competitive under the tested conditions, not that the measurements are universally uninformative. Overprediction could overcommit recovered heat; underprediction could leave heat unused. Signed errors and large misses therefore matter. One facility, one year, missing telemetry, and operating shifts limit generalization. Evaluation is offline; no measured energy savings or automatic control performance is claimed.

3. Related work

Sun et al. (2024) release and document Frontier's power and coolant measurements [1]. This supplies the dataset and physical definitions. The project adds a forecasting experiment; a data descriptor does not establish forecasting accuracy.

Antal et al. (2019) use multilayer perceptrons to approximate CFD-derived hot air temperatures for heat reuse [2]. This demonstrates learned thermal simulation surrogates. Here the target is future measured coolant side heat, not a simulated spatial temperature field.

Jadhav and Liu (2026) use physics guided gradient boosting on Frontier data to predict accessory power [3]. Their preprint reports MAE of 0.026 MW. It motivates joint operating inputs, but its different target and protocol prevent direct error comparisons.

Lahouar and Ben Hadj Slama (2015) combine random forests and expert input selection for dayW ahead electrical load prediction [4]. This supplies an energy forecasting and input selection precedent. Here individual operating variables are tested for next hour waste heat.

Brecelj et al. (2022) compare naive, neural, and boosted tree forecasts of industrial sensors 30/60 minutes ahead [5]. Trees performed best; neural networks did not consistently beat the naive predictor. This motivates persistence and history length comparisons on data center heat.

Ma et al. (2025) forecast Frontier coolant return temperature using Transformer-GRU [6]. They report lower errors than tested alternatives including LSTM, GRU, and CNN-GRU. This project instead predicts next hour heat and tests added variable value; temperature and heat errors are not directly comparable.

1D CNNs learn local history patterns, **LSTMs/GRUs** update gated states, and **Transformers** use attention [6]. Experiments prioritize a small CNN and optional GRU; LSTM and Transformer models remain background alternatives.

4. Data

The *Frontier HPC & Facility Data* workbook was released by Grant and colleagues at Oak Ridge National Laboratory on Figshare, version 4, under **CC BY 4.0** [7]. Facility monitoring is documented by Sun et al. [1]. Source: https://doi.org/10.6084/m9.figshare.24391240.

The workbook contains **49,869 measurements and 18 columns**, including timestamps, covering 2023 at nominal 10-minute intervals. Table 1 describes the selected numerical variables. The regression target is next-hour average `Overall_WasteHeat` in MW; Figure 1 shows its hourly distribution.

Table 1: Original 10-minute measurements across 2023. SD denotes sample standard deviation.

Variable (unit)	Mean	SD	Minimum	Maximum	Missing
Waste heat (MW)	9.09	3.09	0.00	26.34	0
Computing power (MW)	11.60	3.29	0.76	27.70	0
Supply temperature (°C)	19.98	5.45	7.28	33.56	0
Return temperature (°C)	31.20	4.31	7.65	45.62	122
Coolant flow (gallons/min)	3832.13	1356.74	0.00	6364.80	0

The units row is removed. Timestamp offsets up to 0.635 seconds are rounded to the nearest 10-minute mark without creating duplicates. There are 2,691 fewer records than the 52,560 expected; gaps remain explicit. Recorded zeros are retained pending operating-state checks. Sensor noise and nonrandom missingness remain limitations.

Requiring six valid observations per hour for all five variables leaves **8,256 complete hours out of 8,760**, without interpolation. Examples require consecutive complete input and target hours. Input-set and history comparisons use common eligible target hours; every experiment reports its sample count.

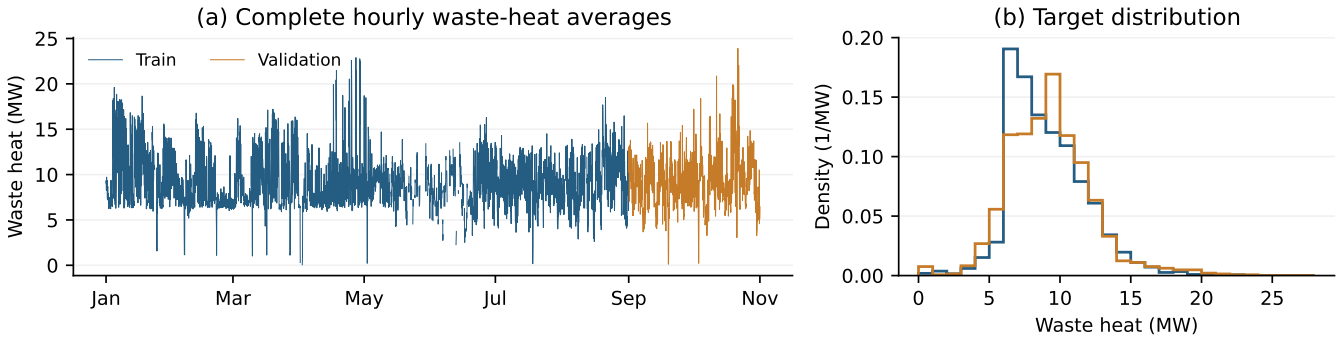


Figure 1: Complete hourly averages and normalized histograms for January-August (training) and September-October (validation).

5. Proposed models and methods

After hour t ends, forecast Q_{t+1} from $x_t = (Q_t, P_t, T_{s,t}, T_{r,t}, F_t)$. Linear models use five features for one hour history or 15 for three hours. Neural models preserve three hours as an ordered 3×5 sequence. No target interval measurements enter inputs.

Core models. Persistence predicts $\widehat{Q}_{t+1} = Q_t$. Linear regression from scratch minimizes training MSE with NumPy gradient descent. A ridge variant penalizes weights, excluding the intercept, to test correlated predictors; scikit-learn provides an independent check [8].

Neural extension. A **1D CNN** learns and cross variable patterns. A**GRU** tests recurrent modeling [6]. Both predict scalar heat from three observed hours; this short history does not support long memory claims. LSTM/Transformer training will be studied if time permites

PyTorch [9] will use MSE, Adam, validation-MAE early stopping, a small hyperparameter search, and three fixed seeds. Scaling uses training data only. A random forest is a fallback [4, 8]. Only persistence has run; learned model results remain future work.

6. Evaluation

I will Train on January-August, validate on September-October, and test on November-December. The target hour determines its split. Earlier inputs may cross a boundary because they are available at forecast time. Targets remain within their assigned periods. There is no random splitting, gap interpolation, or preprocessing fitted outside training. Neural windows retain time order and are stateless across windows. Validation selects settings; test scoring follows model freezing.

Completed persistence baseline: one hour examples use the five variable completeness rule. Persistence has no fitted parameters; training period errors are descriptive.

Period	Examples	MAE (MW)	RMSE (MW)
Training	5,293	1.493	2.197
Validation	1,452	1.430	2.118

Test errors have not been calculated. **MAE** measures absolute error in MW; **RMSE** emphasizes large misses. Signed bias and rapid change errors assess operational consequences. Improvement on identical target hours is $100(1 - \mathrm{MAE}_{model}/\mathrm{MAE}_{persistence})\%$.

Linear ablations compare heat only, heat plus each additional variable separately, and all five inputs. One/three hour histories use common target hours. CNN and optional GRU compare heat only/all five inputs on the same three hour windows as persistence and linear models. No improvement means no benefit in that experiment, not no physical relationship.

Karim Salha's feedback motivates training/validation operating distribution checks and, after freezing, test covariate diagnostics. Monthly/regime errors assess generalization. Arek Kalayjian's feedback motivates individual variable ablations. Any uncertainty intervals resample contiguous time blocks. Published errors with different targets/horizons are contextual, not direct benchmarks.

7. Resources and code

I will start on my laptop or Colab CPU; a Colab GPU is optional for compact neural models if runtime warrants it. AUB HPC allocation is planned. Record runtime, memory, and parameter counts.

I will Use pandas/openpyxl for loading [10, 11], NumPy for calculations [12], Matplotlib for plots [13], PyTorch for neural models [9], and scikit-learn for checks/fallback forest [8]. The public repository will include loading/baseline code, dependencies, settings, seeds, README, and attributable commits. Existing code reproduces exploration and persistence.

8. Timeline

Task owner: Nkezi Emmanuel Akofor.

Dates (2026)	Milestone
18–27 Oct	Finalize data checks and public repository; implement and verify and fitting.
28–3 Nov	Run heat only, individual variable, and ridge comparisons on validation.
04–10 Nov	Compare linear history lengths; train a compact 1D CNN; optional GRU if time permits.
11–17 Nov	Check neural seed variability, errors, and operating shifts; progress report on **17 November**.
18–24 Nov	Freeze models and settings; score test once; compute uncertainty and diagnostic summaries.
25–1 Dec	Draft final report and poster; connect findings to the research question.
2–4 Dec	Reproduce results and finish poster for the **4 December** session.
5–10 Dec	Resolve remaining reproducibility issues and submit final report by **10 December**.

References

[1] Jian Sun, Zhiming Gao, David Grant, Kashif Nawaz, Pengtao Wang, Cheng-min Yang, Philip Boudreaux, Steve Kowalski, and Shean Huff. Energy dataset of Frontier supercomputer for waste heat recovery. *Scientific Data*, 11, 2024. doi: 10.1038/s41597-024-03913-w.

[2] Marcel Antal, Tudor Cioara, Ionut Anghel, et al. Reuse of data center waste heat in nearby neighborhoods: A neural networks-based prediction model. *Energies*, 12(5):814, 2019. doi: 10.3390/en12050814.

[3] Shrenik Jadhav and Zheng Liu. Machine learning guided cooling system optimization for data center. arXiv:2601.02275v3, preprint, 2026.

[4] Ali Lahouar and Jaleleddine Ben Hadj Slama. Day-ahead load forecast using random forest and expert input selection. *Energy Conversion and Management*, 103:1040–1051, 2015. doi: 10.1016/j.enconman.2015.07.041.

[5] Bor Brecelj, Beno Sircelj, Joze M. Rozanec, Blaz Fortuna, and Dunja Mladenic. Forecasting sensor values in waste-to-fuel plants: a case study. arXiv:2209.13957, preprint, 2022.

[6] Shuaiyin Ma, Yuyang Liu, Yang Liu, Jiaqiang Wang, Qiu Fang, and Yuanfeng Huang. Artificial intelligence-enabled predictive energy saving planning of liquid cooling system for data centers. *Advanced Engineering Informatics*, 65: 103283, 2025. doi: 10.1016/j.aei.2025.103283.

[7] David Grant, Zhiming Gao, Steve Kowalski, Jian Sun, Kashif Nawaz, Pengtao Wang, Cheng-min Yang, Philip Boudreaux, and Shean Huff. Frontier HPC & Facility Data. Figshare, version 4. CC BY 4.0, 2024. URL https://figshare.com/articles/dataset/24391240.

[8] Fabian Pedregosa, Gael Varoquaux, Alexandre Gramfort, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL https://jmlr.org/papers/v12/pedregosa11a.html.

[9] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.

[10] Wes McKinney. Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010. doi: 10.25080/Majora-92bf1922-00a.

[11] openpyxl developers. openpyxl documentation, 2026. URL https://openpyxl.readthedocs.io/en/stable/. Accessed 20 October 2026.

[12] Charles R. Harris, K. Jarrod Millman, Stefan J. van der Walt, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[13] John D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi: 10.1109/MCSE.2007.55.